

CompNeuroVis: Connecting Existing Computational Neuroscience Workflows to Interactive Applications

Orren Ravid
Independent Researcher

The Problem

Many computational neuroscience workflows depend on a large variety of simulators and tools. Current options for visualizing and interacting with models are often tied to a particular simulator, model platform, or model tuning environment. The alternative is generic scientific plotting libraries that don't offer the same advanced capabilities as these other native interfaces.

Our Approach

CompNeuroVis is a Python toolkit for building interactive applications around the existing computational neuroscience workflows of each user. It provides a compositional style similar to common scientific plotting libraries, while offering many of the advanced visualization and interactive capabilities that are often otherwise only available through simulator-, platform-, or environment-specific tools.

Your Model

cnv.
source()

cnv.
show()

Add
Widgets

```
import compneurovis as cnv

model = [your model generation code]

sim = cnv.source(lambda ctx: model.step())

morphology = sim.morphology(model.morphology, ...)

voltage_plot = sim.line(source=morphology.selection,
                        variables={"Voltage": "v"}, ...)

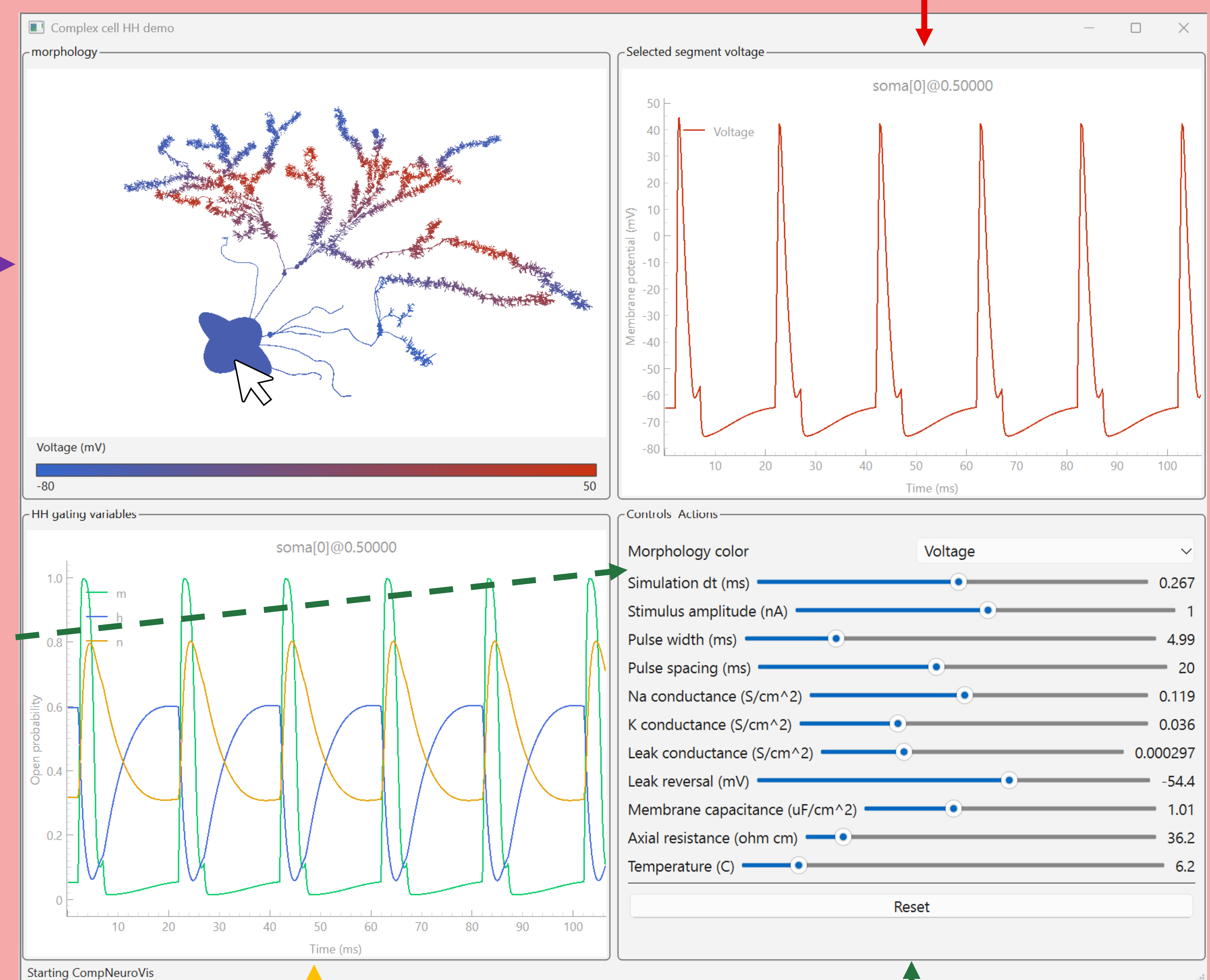
gates_plot = sim.line(source=morphology.selection,
                     variables=HH_GATE_VARIABLES, ...)

sim.control("morphology_color", ..., kind="dropdown", ...)
sim.control("dt", ..., get=model.get_dt,
            set=model.set_dt, ...)

...

cnv.layout(((morphology, voltage_plot),
            (gates_plot, sim.controls_panel)))

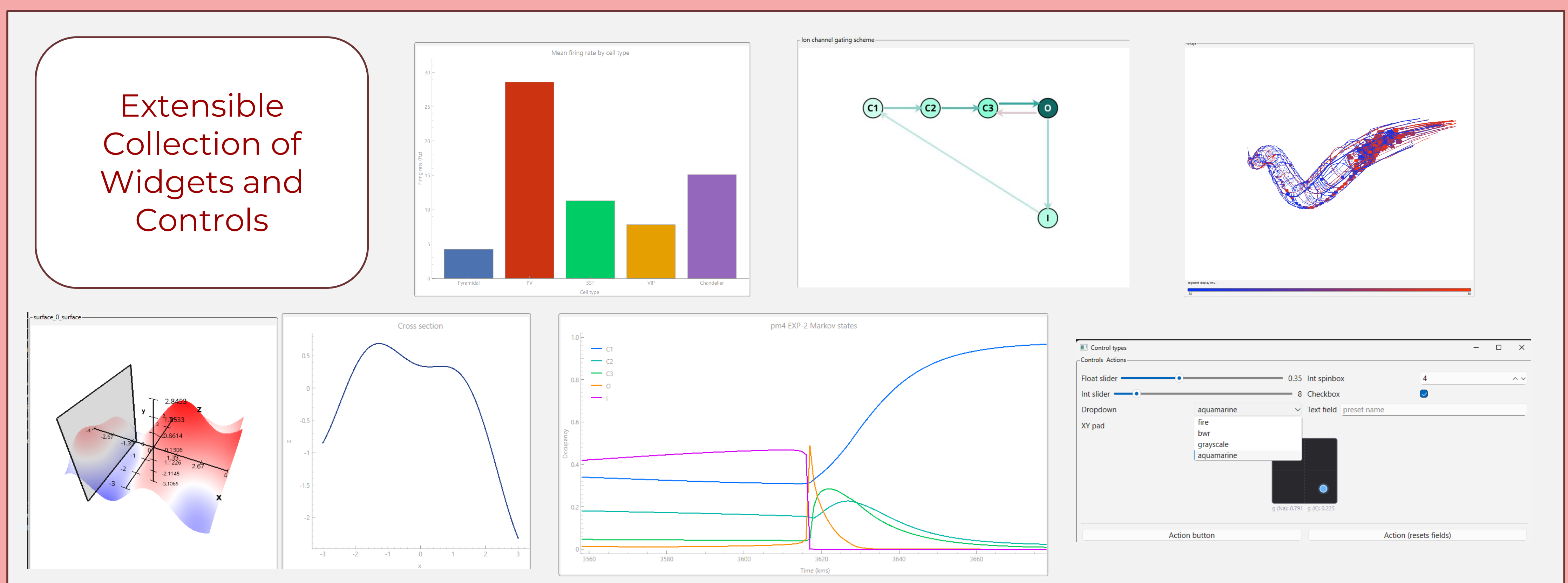
cnv.show(title="Complex cell HH demo")
```



Support for Multiple Simulators as Well as Custom Python Implementations

```
# NEURON
sim = cnv.neuron.source(...)
# Jaxley
sim = cnv.jaxley.source(...)
# Custom
sim = cnv.source(...)
```

Extensible Collection of Widgets and Controls



References

- Hines, M. L., & Carnevale, N. T. (1997). The NEURON simulation environment. *Neural Computation*, 9(6), 1179-1209. <https://doi.org/10.1162/neco.1997.9.6.1179>
- Cantarelli, M., et al. (2018). Geppetto: A reusable modular open platform for neuroscience data and models. *Philosophical Transactions B*, 373(1758), 20170380.
- Dura-Bernal, S., et al. (2019). NetPyNE: a tool for data-driven multiscale modeling of brain circuits. *eLife*, 8, e44494.
- Makarov, R., Chavlis, S., & Poirazi, P. (2025). DendroTweaks: An interactive approach for unraveling dendritic dynamics. *eLife*, 13, RP103324.
- Deistler, M., Kadhimi, K. L., Pals, M., Beck, J., Huang, Z., Gloeckler, M., Lappalainen, J. K., Schröder, C., Berens, P., Gonçalves, P. J., & Macke, J. H. (2025). Jaxley: Differentiable simulation enables large-scale training of detailed biophysical models of neural dynamics. *Nature Methods*, 22, 2649-2657. <https://doi.org/10.1038/s41592-025-02895-w>
- Ray, S., & Bhalla, U. S. (2008). PyMOOSE: Interoperable scripting in Python for MOOSE. *Frontiers in Neuroinformatics*, 2, 6. <https://doi.org/10.3389/neuro.11.006.2008>
- Abi Akar, N., Cumming, B., Karakasis, V., Küsters, A., Klijn, W., Peyser, A., & Yates, S. (2019). Arbor: A morphologically-detailed neural network simulation library for contemporary high-performance computing architectures. 2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP), 274-282. <https://doi.org/10.1109/EMDPDP.2019.8671560>

Acknowledgements: I thank Ethan Irby (Research Assistant, Nathan Kline Institute) for ideas on use cases and capability requirements, and the open-source computational neuroscience community and simulator developers whose tools and discussions informed this work.

Contact:

✉ orren.ravid@umassmed.edu

🌐 <https://github.com/orrenravid/>

🌐 <https://www.linkedin.com/in/orren-ravid/>